

Practically Groovy: Groovy's growth spurt

Get friendly with the new, JSR-compliant Groovy syntax

Skill Level: Intermediate

[Andrew Glover](mailto:andrew@thirstyhead.com) (andrew@thirstyhead.com)

Co-Founder

ThirstyHead.com

[Scott Davis](mailto:scott@thirstyhead.com) (scott@thirstyhead.com)

Founder

ThirstyHead.com

19 Jul 2005

With the release (and subsequent releases) of a JSR-241 compliant parser, the changes to Groovy's syntax have been formalized -- which means if you weren't paying attention before, now's the time to start. This month, resident Groovy practitioner Andrew Glover walks through most important changes to Groovy's syntax and shows you a handy feature you won't find in classic Groovy.

It's been almost a year since I introduced you to Groovy with the article "[Feeling Groovy](#)" in the *alt.lang.jre* series. Since then, Groovy has matured quite a bit through a number of releases that have progressively addressed problems in the language implementation and feature requests from the developer community. Finally, Groovy took a gigantic leap this past April, with the formal release of a new parser aimed at standardizing the language as part of the JSR process.

In this month's installment of *Practically Groovy*, I'll celebrate the growth of Groovy by introducing you to the most important changes formalized by Groovy's nifty new parser; namely variable declarations and closures. Because I'll be comparing some of the new Groovy syntax to the classic syntax found in my first-ever article on Groovy, you may want to open up "[Feeling Groovy](#)" in a second browser window now.

Why change things?

If you've been following Groovy for any amount of time, whether you've been reading articles and blogs or writing code yourself, you may have gotten wind of one or two subtle issues with the language. When it came to clever operations such as object navigation, and particularly closures, Groovy suffered from occasional ambiguities and an arguably limiting syntax. Some months ago, as part of the JSR process, the Groovy team began working on resolving these issues. The solution, presented in April with the release of groovy-1.0-jsr-01, was an updated syntax and a new-syntax-savvy parser to standardize it.

About this series

The key to incorporating any tool into your development practice is knowing when to use it and when to leave it in the box. Scripting languages can be an extremely powerful addition to your toolkit, but only when applied properly to appropriate scenarios. To that end, [Practically Groovy](#) is a series of articles dedicated to exploring the practical uses of Groovy, and teaching you when and how to apply them successfully.

The good news is that the new syntax is chock full of enhancements to the language. The other good news is that it isn't that drastically different from the old. Like all of Groovy, the syntax was designed for a short learning curve and a big payoff.

Of course, the JSR-compliant parser has rendered some now "classic" syntax incompatible with the new Groovy. You can see this for yourself if you try running a code sample from an early article in this series with the new parser: It probably won't work! Now, this may seem a little strict -- especially for a language as freewheeling as Groovy -- but the point of the parser is to ensure the continued growth of Groovy as a *standardized* language for the Java platform. Think of it as a helpful tour guide to the new Groovy.

Hey, it's still Groovy!

Before getting too far into what's changed, I'll take a second to chat about what *hasn't*. First, the basic nature of dynamic typing hasn't changed. Explicit typing of variables (that is, declaring a variable as a `String` or `Collection`) is still optional. I'll discuss the one slight addition to this rule shortly.

Many will be relieved to know that semicolons are also still optional. Arguments were made for and against this syntactic leniency, but in the end the less-is-more crowd won the day. Bottom line: You are still free to use semicolons if you want to.

Collections have also stayed the same for the most part. You can still declare list-like collections using the *array* syntax and maps the same way you always have (that is, the way you first learned in "[Feeling Groovy](#)"). Ranges, on the other hand, have changed slightly, as I'll soon demonstrate.

Finally, the Groovy additions to standard JDK classes haven't changed a bit. Syntactic sugar and nifty APIs are intact, as in the case of normal-Java File types, which I'll show you later.

Variably variables

The rules on Groovy variables have probably taken the hardest hit with the new JSR-compliant syntax. Classic Groovy was quite flexible (and indeed terse) when it came to variable declarations. With the new JSR Groovy, all variables must be preceded with either the `def` keyword or a modifier such as `private`, `protected`, or `public`. Of course, you can always declare the variable type as well.

For example, when I introduced you to *GroovyBeans* in "[Feeling Groovy](#)," I defined a type called `LavaLamp` in that article's Listing 22. That class is no longer JSR compliant and will result in parser errors if you try to run it. Fortunately, migrating the class isn't hard: All I had to do is add the appropriate data types to all my desired members, as shown in Listing 1 below:

Listing 1. Return of the LavaLamp

```
package com.vanward.groovy

class LavaLamp{
    int model
    String baseColor
    String liquidColor
    String lavaColor
}

def llamp = new LavaLamp(model:1341,
                        baseColor:"Black",
                        liquidColor:"Clear",
                        lavaColor:"Green")

println llamp.baseColor
println "Lava Lamp model ${llamp.model}"

def myLamp = new LavaLamp()
myLamp.baseColor = "Silver"
myLamp.setLavaColor("Red")
println "My Lamp has a ${myLamp.baseColor} base"
println "My Lava is " + myLamp.getLavaColor()
```

Not so bad, right?

As described above, the `def` keyword is required for any variable that doesn't otherwise have a modifier or type. For example, the code in Listing 2 contains the

spurious `numstr` variable in the `toString` method, which will result in an error if run with the JSR parser:

Listing 2. Don't forget the `def` keyword!

```
class Person {
    String fname
    String lname
    int age
    String address
    List contactNumbers = []

    String toString(){
        //NOTE: the next line is not valid
        //      because it doesn't start with
        //      a "def" or a data type
        numstr = new StringBuffer()
        if (contactNumbers != null){
            contactNumbers.each{
                numstr.append(it)
                numstr.append(" ")
            }
        }
        return "first name: ${fname} " +
            "last name: ${lname} " +
            "age: ${age} " +
            "address: ${address} " +
            "contact numbers: ${numstr.toString()}"
    }
}
```

Recognize that code? It's borrowed from Listing 1 of "[Stir some Groovy into your Java apps](#)." In Listing 3, you can see the error message that will pop up if you try to run the code as is:

Listing 3. Whoops -- nice error message!

```
c:\dev\projects>groovy BusinessObjects.groovy

BusinessObjects.groovy: 13: The variable numstr is undefined in the current scope
@ line 13, column 4.
    numstr = new StringBuffer()
    ^
1 Error
```

The solution, of course, is to add the `def` keyword to `numstr` in the `toString` method. This rather *deft* solution is shown in Listing 4.

Listing 4. A `def(t)` remix

```
class Person {
    String fname
    String lname
    int age
    String address
```

```

List contactNumbers = []

String toString(){
    def numstr = new StringBuffer()
    if (contactNumbers != null){
        contactNumbers.each{
            numstr.append(it)
            numstr.append(" ")
        }
    }
    return "first name: ${fname} " +
        "last name: ${lname} " +
        "age: ${age} " +
        "address: ${address} " +
        "contact numbers: ${numstr.toString()}"
}
}

```

Alternatively, I could have declared `numstr` as a `StringBuffer`. Either way, the point is that in JSR Groovy, I have to precede the variable with *something*.

Closing in on closures

The syntax for closures has changed, but mostly only with regard to parameters. In classic Groovy, if you declared a parameter to your closure you had to use a `|` character for a separator. As you probably know, `|` is also a bitwise operator in normal Java language; consequently, in Groovy, you couldn't use the `|` character unless you were in the context of a parameter declaration of a closure.

You saw classic Groovy parameter syntax for closures in Listing 21 of "[Feeling Groovy](#)," where I demonstrated iteration. As you'll recall, I utilized the `find` method on collections, which attempted to find the value 3. I passed in the parameter `x`, which represents the next value of the iterator (experienced Groovy developers will note that `x` is entirely optional and I could have referenced the implicit variable `it`). With JSR Groovy, I must drop the `|` and replace it with the Nice-ish `->` separator, as shown in Listing 5 below:

Listing 5. New Groovy closure syntax

```

[2, 4, 6, 8, 3].find { x ->
    if (x == 3){
        println "found ${x}"
    }
}

```

Doesn't the newer closure syntax remind you of the Nice language's block syntax? If you are not familiar with the Nice language, check out [Twice as Nice](#), another of my contributions to the *alt.lang.jre* series.

As I mentioned earlier, Groovy's JDK hasn't changed. But as you've just learned, closures have; therefore, the way you use those nifty APIs in Groovy's JDK have

also changed, but just slightly. In Listing 6, you can see how the changes impact Groovy IO; which is hardly at all:

Listing 6. Groovy's JDK is still powerful!

```
import java.io.File

new File("maven.xml").eachLine{ line ->
    println "read the following line -> " + line
}
```

Reworking filters

Now, I hate to make you skip around a lot, but remember how in "[Ant Scripting with Groovy](#)" I spent some time expounding on the power and utility of closures? Thankfully, much of what I did on the examples for that column is easy to rework for the new syntax. In Listing 7, I simply add a `def` attribute onto the `strategy` member of `Filter`, which was originally shown in Listings 2 and 3 of that article. I then add the `->` separator in my closures, and *voilà* -- it works!

Listing 7. Filtering reworked!

```
class Filter{
    def strategy
    boolean apply(String str){
        return strategy.call(str)
    }
}

def simplefilter = { str ->
    if(str.indexOf("java.") >= 0){
        return true
    }else{
        return false
    }
}

def fltr = new Filter(strategy:simplefilter)
assert !fltr.apply("test")
assert fltr.apply("java.lang.String")

def rfilter = { istr ->
    if(istr =~ "com.vanward.*"){
        return true
    }else{
        return false
    }
}

def rfltr = new Filter(strategy:rfilter)
assert !rfltr.apply("java.lang.String")
assert rfltr.apply("com.vanward.sedona.package")
```

So far so good, don't you think? The new Groovy syntax is quite easy to pick up!

Changes to ranges

Groovy's range syntax has changed ever so slightly. In classic Groovy, you could get away with using the `...` syntax to denote exclusivity, that is, the upper bound. In JSR Groovy, you'll simply drop that last dot (.) and replace it with the intuitive `<` symbol.

Watch as I rework my range example from "Feeling Groovy" in Listing 8 below:

Listing 8. New range syntax

```
def myRange = 29..
```

Ambiguous, you say?

You may have noticed, while playing with Groovy, a subtle feature that lets you obtain a reference to a method and invoke that reference at will. Think of the method pointer as a short-hand convenience mechanism for invoking methods along an object graph. The interesting thing about method pointers is that their use can be an indication that the code violates the Law of Demeter.

"What's the Law of Demeter," you say? Using the motto *Talk only to immediate friends*, the Law of Demeter states that we should avoid invoking methods of an object that was returned by another object's method. For example, if a Foo object exposed a Bar object's type, clients could access behavior of the Bar *through* the Foo. The result would be brittle code, because changes to one object would ripple through a graph.

A respected colleague wrote an excellent article entitled "The Paperboy, the Wallet, and the Law of Demeter" (see [Resources](#)). The examples in the article are written in the Java language; however, I've redefined them below using Groovy. In Listing 9, you can see how this code demonstrates the Law of Demeter -- and how it could be used to wreak havoc with people's wallets!

Listing 9. Demeter in action (tsk, tsk!)

```
package com.vanward.groovy
import java.math.BigDecimal
```

```
class Customer {
    String firstName
    String lastName
    Wallet wallet
}

class Wallet {
    def value;
    def getTotalMoney() {
        return value;
    }

    def setTotalMoney(newValue) {
        value = newValue;
    }
    def addMoney(deposit) {
        value = value.add(deposit)
    }
    def subtractMoney(debit) {
        value = value.subtract(debit)
    }
}
```

In Listing 9 there are two defined types -- a Customer and a Wallet. Notice how the Customer type exposes its own wallet instance. As previously stated, the code's naive exposures present issues. For example, what if I (as the original article's author did) added in an evil paperboy to ravage unsuspecting customer wallets? I've used Groovy's method pointers for just this nefarious purpose in Listing 10. Note how I am able to grab a reference to the subtractMoney method via an instance of Customer with Groovy's new & syntax for method pointers.

Listing 10. Enter the paperboy ...

```
def iwallet = new Wallet(value:new BigDecimal(32))
def victim = new Customer(firstName:"Lane", lastName:"Meyer", wallet:iwallet)
//Didn't *ask* for a dime. Two Dollars.
victim.getWallet().subtractMoney(new BigDecimal("0.10"))

//paperboy turns evil by snatching a reference to the subtractMoney method
def mymoney = victim.wallet.&subtractMoney
mymoney(new BigDecimal(2)) // "I want my 2 dollars!"
mymoney(new BigDecimal(25)) // "late fees!"
```

Now, don't get me wrong: Method pointers aren't meant for hacking into code or obtaining references to people's cash! Rather, a method pointer is a convenience mechanism. Method pointers are also great for reconnecting with your favorite 80s movies. They can't help you if you get those lovable cute furry things wet, though! In all seriousness, think of Groovy's `println` shortcut as an implicit method pointer to `System.out.println`.

If you were paying careful attention you will have noted that JSR Groovy requires me to use the `new &` syntax to create a pointer to the method `subtractMoney`. This addition, as you've probably guessed, clears up ambiguities in classic Groovy.

And here's something new!

It wouldn't be fun if there wasn't anything *new* in Groovy's JSR releases, would it? Thankfully, JSR Groovy has introduced the `as` keyword, which is a short-hand casting mechanism. This feature goes hand-in-hand with a new syntax for object creation, which makes it easy to create *non-custom* classes in Groovy with an array-like syntax. By non-custom, I mean classes found in the JDK such as `Color`, `Point`, `File`, etc.

In Listing 11, I've used the new syntax to create some simple types:

Listing 11. No new features in Groovy? 'as' if!

```
def nfile = ["c:/dev", "newfile.txt"] as File
def val = ["http", "www.vanwardtechnologies.com", "/"] as URL
def ival = ["89.90"] as BigDecimal
println ival as Float
```

Note that I created a new `File` and `URL`, as well as a `BigDecimal` using the short-hand syntax, as well as how I was able to cast the `BigDecimal` type to a `Float` using `as`.

What's next?

See [Resources](#) to download the latest release and learn more about the JSR Groovy process, and tune in to *Practically Groovy* next month, when I (and two guest columnists) will delve into the finer details of Groovy closures.

About the authors

Andrew Glover

Andrew Glover is a developer, author, speaker, and entrepreneur. He is the founder of the [easyb](#) Behavior-Driven Development (BDD) framework and is the co-author of three books: [Continuous Integration](#), [Groovy in Action](#), and [Java Testing Patterns](#). He teaches a wide variety of Groovy-, Grails-, and testing-related classes at [ThirstyHead.com](#). You can keep up with Andy at [thediscoblog.com](#) where he routinely blogs about software development.

Scott Davis

Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of ThirstyHead.com, a Groovy and Grails training company. His books include [Groovy Recipes: Greasing the Wheels of Java](#), [GIS for Web Developers: Adding Where to Your Application](#), [The Google Maps API](#), and [JBoss At Work](#). He writes two ongoing article series for IBM developerWorks: [Mastering Grails](#) and [Practically Groovy](#).